PROJECT REPORT
AI-VIRTUAL-ASSISTANT

A cross-platform personal AI assistant with real-time voice interaction, screen vision, system control, browser automation, and persistent memory.

Project Title	AI-VIRTUAL-ASSISTANT (MARK LVI)
Student / Owner	Tejas Gowda M
Repository	github.com/tejas448866/AI-VIRTUAL-ASSISTANT
Platform	Windows, macOS, Linux (Python desktop)

1. Project Purpose

AI-VIRTUAL-ASSISTANT is a comprehensive autonomous desktop assistant. It combines real-time voice conversation, visual awareness of the screen and webcam, system-level control, web browsing, file and code operations, reminders, and messaging into a single interactive application.

The system is designed for users who want a capable local companion that can hear, see, speak, and act on the computer. Primary goals include low-latency voice interaction, honest tool feedback (done / unverified / failed), optional fully local speech and language models, and a plugin architecture that lets new skills be added as single files.

Target users are developers, power users, and students exploring agentic AI, computer-use automation, and multi-modal assistants. The project emphasises cross-platform support, privacy-conscious local options, and clear confirmation for irreversible actions.

2. Technologies Used

The following technologies appear in the repository source and requirements:

- **Python 3** — main language for the application and tools.
- **PyQt6** — desktop GUI, holographic avatar rendering (software QPainter), HUD, and settings panels.
- **google-genai** — Gemini Live API for real-time bidirectional audio and tool calling.
- **sounddevice + NumPy** — microphone capture, speaker playback, and audio-level / viseme analysis.
- **OpenCV + Pillow + mss** — screen capture, webcam frames, and image processing for vision tools.
- **Playwright** — browser automation (open URLs, navigate, interact).
- **requests, BeautifulSoup, yt-dlp, ddgs** — web search, page parsing, and media extraction.
- **psutil, pyautogui, pyperclip** — system monitoring and desktop input control.
- **FastAPI + uvicorn** — optional remote dashboard (phone control via QR pairing).
- **Optional local stack** — faster-whisper (STT), piper-tts (TTS), Ollama / OpenAI-compatible APIs for fully local or third-party engines.

Platform-specific packages (e.g. pycaw, pywin32, win10toast on Windows) are gated by environment markers so a single requirements.txt works on Windows, macOS, and Linux.

3. Working and Logic

High-Level Workflow

1. The application starts a PyQt6 HUD with an animated holographic avatar and audio waveform.
2. Audio is streamed to Gemini Live (or a configured local STT → LLM → TTS pipeline).
3. The model receives a live tool catalogue discovered from actions/ and plugins/ at startup.

4. When the user requests an action, the corresponding tool runs and returns a structured outcome (done / unverified / failed).

5. Screen or camera frames can be captured and labelled for vision-aware replies.

6. Persistent memory stores facts and session summaries for later recall; an undo stack can reverse file and settings changes.

7. Wake-word mode (optional) keeps the mic closed until “Hey Jarvis” is detected; push-to-talk is also supported.

Core Capabilities

• **Real-time voice** — ultra-low latency conversation via Gemini Live; optional local Whisper + Piper in ~50 languages.

• **Visual awareness** — screen capture and webcam frames injected into the session.

• **System control** — launch/close apps, volume, brightness, Wi-Fi, power actions with confirmation gates.

• **Browser & search** — open URLs, navigate, multi-mode web search (news, research, price, compare).

• **Files & code** — read, summarise, write, and process local documents; code writing and debugging support.

• **Reminders & messaging** — OS-native scheduled notifications; compose messages for WhatsApp, Telegram, and similar.

• **Hardware monitoring** — continuous CPU, RAM, GPU, and temperature telemetry with voice alerts.

• **Plugin system** — drop a single .py file into plugins/ to add a skill on next launch.

Main Components

• **main.py** — session loop, audio I/O, Gemini Live connection, stall detection, viseme extraction.

• **ui.py (JarvisUI)** — PyQt6 HUD, avatar, settings drawers (engine, providers, wake word, devices).

• **core/** — action loader, plugin loader, memory, config, undo, confirm gate, echo guard, wake word.

• **actions/** — self-describing tools (screen processor, system monitor, web search, browser control, etc.).

• **memory/** — persistent facts, session summaries, and on-demand search.

• **providers/** — optional STT / LLM / TTS backends (only loaded when chosen in the UI).

• **setup.py / run.bat** — dependency install and launch helpers.

Output

The system produces spoken replies, on-screen HUD status, structured tool results, optional remote-dashboard control, session memory, and diagnostic feedback (audio levels, connection health, outcome codes). Irreversible actions require explicit user confirmation.

4. Testing and Limitations

The repository provides setup scripts and detailed documentation for manual verification. End-to-end behaviour depends on a valid Gemini API key (or configured local providers), working microphone/speakers, and OS permissions for screen capture and input control.

Challenges Observed

• Live sessions can stall without closing the socket; a turn-scoped timeout recovers the conversation.

• OS permission models differ (especially screen capture and accessibility on macOS / Wayland).

• Cloud quota limits on free tiers affect background text tools differently from the voice stream.

• Self-echo and room acoustics require careful audio gating so the assistant does not answer itself.

Limitations

• Full capability requires either Gemini Live access or a correctly configured local STT/LLM/TTS stack.

• Some plugins need extra packages (Gmail OAuth, Tuya, MQTT, etc.) that are optional at install time.

• Desktop automation is inherently fragile across UI themes and OS updates.

• Local neural STT/TTS models add significant disk and download size if enabled.

Error Handling & Safety

Tool results are reported as done, unverified, or failed rather than assumed successful. Shutdown, restart, and similar actions wait for an explicit user button press. An undo stack covers file and settings changes. Untrusted web or document content is wrapped so instructions inside it are not obeyed as commands. Missing calibration or optional modules fall back gracefully without crashing the main session.

Possible Future Improvements

- Broader plugin library and clearer community contribution guidelines.
- Stronger automatic adaptation to OS permission failures with clearer user guidance.
- Expanded offline-first defaults for environments that cannot use cloud APIs.
- Improved single-command multi-step planning with better progress feedback.

5. GitHub and Originality

Repository Structure

Key items present in the repository include:

- README.md / readme.md — project description and detailed capability documentation
- main.py, ui.py, setup.py — entry point, GUI, and installer
- requirements.txt — cross-platform dependency list with optional sections
- run.bat / run_mark_lvi.bat — Windows launch helpers
- core/, actions/, memory/, providers/, plugins/ — modular application packages
- LICENSE, .gitignore, VirtualAIBotJ1.zip — licensing and distribution assets

Execution Steps

1. Install a supported Python version and clone the repository.
2. Run `python setup.py` (or `pip install -r requirements.txt`).
3. Configure API keys (Gemini and any optional providers) via the config / UI panels.
4. Launch with `python main.py` or the provided batch scripts.
5. Grant OS permissions for microphone, speakers, and (if needed) screen capture / accessibility.
6. Optionally enable wake word, local STT/TTS, or remote dashboard from the settings drawer.

Individual Contributions

I, Tejas Gowda M, contributed to the overall project development, integration of the vision and system-control pipeline, configuration, user interface, documentation, and packaging for academic submission.

AI Assistance

AI tools were used for coding assistance, architecture understanding, and preparing documentation during development and report writing.

Learning Outcomes

- Real-time multi-modal interaction with streaming audio and tool calling.
- Designing honest, verifiable computer-use tools with confirmation and undo.
- Building a modular plugin architecture and cross-platform dependency strategy.
- Combining cloud Live APIs with optional fully local STT / LLM / TTS backends.
- Importance of echo guards, stall detection, and clear outcome reporting for reliability.

Conclusion

AI-VIRTUAL-ASSISTANT demonstrates a complete pipeline from real-time voice and vision input to system-level actions and persistent memory. By combining a responsive PyQt6 HUD, Gemini Live (or configurable local engines), self-describing tools, and safety-oriented outcome reporting, the project provides a practical and educational example of an agentic desktop assistant. The modular design, optional local stack, and cross-platform packaging make it both a useful daily companion and a clear demonstration of applied AI for

interactive computer control.

— End of Report —